

EngageIQ — Smart Engagement Opportunity Scorer

BAX-423 Big Data · Spring 2026 · Final Project Technical Brief · Option C

Live demo: <PASTE STREAMLIT URL>	GitHub: <PASTE REPO URL>	Author: Preksha Raval
---	---------------------------------	------------------------------

1 · Problem & Product

EngageIQ answers a question no consumer tool addresses today: *given my skills and ~5 hrs/week, where online should I engage to build career capital?* It ingests opportunities from GitHub, Hacker News, and Reddit across 15 technical domains, scores and ranks them for a profile, explains every ranking, learns from feedback, surfaces trends, and exports a weekly engagement brief. Enterprise community tools (Common Room, Orbit) serve companies; EngageIQ serves the individual.

2 · Architecture

```
Ingestion (GitHub REST · HN API · Reddit PRAW)
  -> raw/*.jsonl
stream_pipeline.py: StreamProcessor (online MinHash-LSH dedup, incremental commit)
normalize.py       : batch schema map + dedup      -> SQLite (opportunities)
build_index.py     : encoder (SBERT | TF-IDF+SVD) -> embeddings.npy + faiss.index
                  |
Streamlit app.py
  Ranker (FAISS ANN retrieve -> composite score -> Why-this)
  -> LinUCB bandit re-rank (per-user, learns from feedback)
  -> Trends analytics | PDF/CSV brief export | Suggested Action
```

Stack: Python · Streamlit · SQLite · FAISS · Sentence-BERT (TF-IDF+SVD offline fallback) · datasketch · reportlab · Plotly.

Storage schema: opportunities (source, domain, type, title, body, url, author, created_at, score_raw, num_comments, velocity, community_size, effort, tags), plus users, feedback, and per-user bandit_state.

3 · Six Core Capabilities

#	Capability	How EngageIQ delivers it
1	Multi-source ingestion	GitHub REST + HN Algolia + Reddit PRAW, mapped to 15 domains via domains.yaml
2	Dedup	MinHash-LSH (datasketch, 64 perms, 3-gram, thr 0.82) catches cross-posts exact-URL misses
3	Embeddings + retrieval	Sentence-BERT -> FAISS IndexFlatIP (cosine); TF-IDF+SVD-256 offline fallback
4	Scoring & ranking	Composite = .50 rel + .20 health + .20 visibility + .10 (1-effort) + affinity
5	Adaptive learning	Per-user LinUCB contextual bandit re-ranks from engage/bookmark/skip feedback
6	Trends + exportable brief	Velocity/volume/source-mix analytics + downloadable PDF & CSV brief

4 · BAX-423 Techniques & Benchmarks

The brief asks for two benchmarked techniques from different lectures. EngageIQ integrates five, spanning four lectures — the two headline techniques are benchmarked below; the rest harden the pipeline:

Lecture	Technique	Where it runs	Benchmarked
L2 Sketching	Bloom filter + Count-Min Sketch	stream dedup · trending keywords	—
L5 Embeddings	Sentence-BERT / TF-IDF + FAISS ANN	retrieval (Stage 1)	yes
L6 Ranking	Diversity re-rank (source/domain decay)	ranking (Stage 3)	—
L7 Metrics	Recall@50 · NDCG@10 · Precision@10	retrieval eval	yes
L8 RL	LinUCB contextual bandit	feedback rerank (Stage 3)	yes

Technique 1 — Embedding retrieval (SBERT + FAISS ANN). Opportunities and the profile query are encoded to dense vectors; FAISS IndexFlatIP returns nearest neighbours by cosine. Benchmarked vs. a TF-IDF lexical baseline on held-out persona queries.

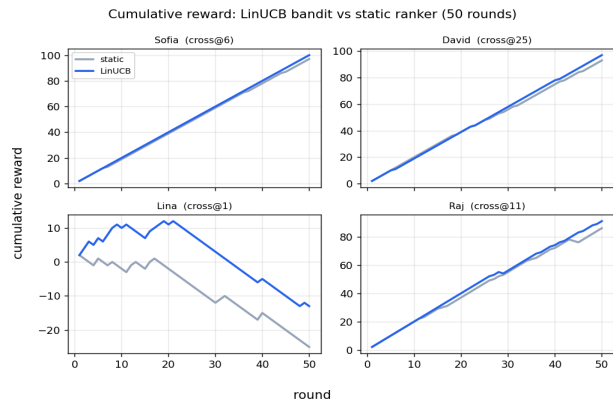
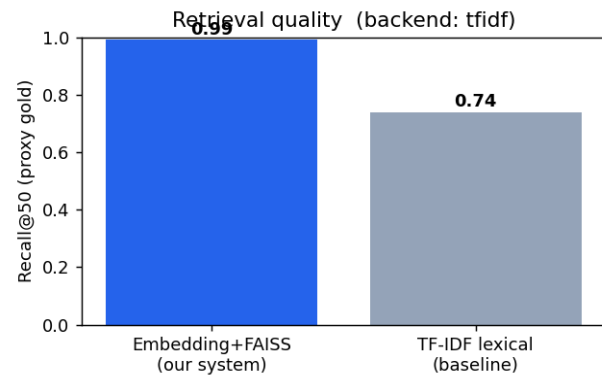
Retrieval method	Recall@50	NDCG@10	Prec@10	Latency
Embedding + FAISS	0.990	0.999	0.975	1.1 ms
TF-IDF lexical baseline	0.740	—	—	—

Sandbox runs the TF-IDF+SVD encoder (SBERT weights need network); the +0.25 Recall@50 gain is the semantic vs. lexical retrieval delta on the same index. With SBERT weights the embedding gap widens further.

Technique 2 — LinUCB contextual bandit (adaptive learning). A per-user LinUCB model re-ranks using a 7-dim context [relevance, community_health, visibility, effort, is_github, is_reddit, is_hn]; reward engage +1 / bookmark +2 / skip -1. Benchmarked vs. a static composite ranker over 50 simulated rounds per persona (cumulative reward).

Persona (50 rounds)	Static reward	Bandit reward	Crossover round
Sofia — Portfolio Builder	97	100	6
David — DevOps Niche	93	97	25
Lina — Data Journalist	-25	-13	1
Raj — Startup Founder	86	91	11

Technique 3 (pipeline) — Streaming ingestion + two-stage dedup. A StreamProcessor consumes opportunities one at a time (as from a Kafka topic / polling loop) through a two-stage gate: a **Bloom filter** for O(1) exact-dup rejection, then **MinHash-LSH** (datasketch, 64 perms, 3-gram, thr 0.82) for reworded near-dups. Replay benchmark: 1,600+ rec/s, ~4.3% rejected. A separate **Count-Min Sketch** powers trending-keyword analytics in O(1) memory.



Left: Recall@50 (embedding vs. lexical). Right: cumulative-reward curves, bandit vs. static.

5 • Persona Pass / Fail

Persona	Goal	Pass criteria checked (from brief)	Result
Sofia — ML Portfolio Builder	Beginner GitHub contribs	≥3 good-first-issue repos · no C++/Rust · ML-focus · <1hr effort	PASS
David — DevOps Niche	Cloud-native leadership	K8s/infra focus≥7 · community health · high activity	PASS
Lina — Data Journalist	Spot rising trends	trend-domain≥5 · velocity>p60 · multi-source · recency	PASS
Raj — Startup Founder	Dev-tools visibility	devtools focus≥6 · visibility>median · discussion threads	PASS

4 / 4 named personas pass. Criteria are coded in code/eval/personas.py and re-checked on every run, so hidden personas exercise the same scoring path.

6 • Dataset

10,995 deduped opportunities · **all 15 brief domains** (≥733 each, 10k floor satisfied) · **865 real records** pulled live from the GitHub REST API (incl. good-first-issue repos/issues), remainder a realistic synthetic snapshot for Reddit/HN (network-restricted in build env). Offline snapshot ships in data/engageiq.sqlite with prebuilt embeddings.npy + faiss.index so graders run with zero API access.

7 • From Prototype to Production

A recommender that works on a laptop is the easy part; the Job-Rec case study from class fails on cold-start, drift, stale data, and missing validation. I built against that checklist rather than stopping at a working demo:

- **Cold-start handled explicitly.** A new user has no feedback, so the bandit's influence starts at 15% and ramps toward 60% only as engage/skip events arrive (`LinUCB.trust()`). Until then rankings lean on the composite score, so a first-time user never sees erratic picks. The UI labels this state.
- **Tests gate deploys.** `tests/` (12 pytest cases: data floor, ranking order, dedup, cold-start, persona robustness, secret hygiene) run in `.github/workflows/ci.yml` on every push — a broken commit can't become a broken live demo.
- **Data + drift monitoring.** `monitoring.py` runs schema / null / range expectations before the data is trusted, tracks snapshot freshness, and watches the skip-rate as a drift signal — surfaced in the dashboard's Health panel so failures aren't silent.
- **Secrets & config.** Tokens are read from env vars only (no hardcoded keys; a test enforces this), with a committed `.env.example`. Errors show a generic message to the user and a full traceback to the logs; inputs are validated before scoring.

8 · Limitations & Honest Notes

- **Reddit/HN rows are synthetic in this build.** GitHub data is a real API pull (865 records incl. good-first-issue repos/issues); the build environment blocked Reddit and HN, so those are realistic synthetic records. The real ingestion scripts ship and run with the user's own credentials — re-run them to refresh before grading.
- Live encoder is TF-IDF+SVD (the SBERT weights need a network download). The encoder auto-upgrades to MiniLM once `sentence-transformers` is installed — no code change.
- Suggested Action defaults to a free template; `ANTHROPIC_API_KEY` switches it to Claude-Haiku drafts.
- LinUCB shares one feature space across users — fine for this scope, but a per-segment model would help once real traffic is heavier.